



Evidencia: Avance 1

Miguel Angel Ramírez Montiel

Universidad Tecmilenio

Computación en Java

Jesús Cazarez Martínez

Jueves 28 de mayo de 2026

1. Ambiente de Desarrollo

Herramientas seleccionadas

Herramienta	Propósito
JDK (OpenJDK)	Compilación y ejecución de Java
IntelliJ IDEA	IDE principal de desarrollo
Git	Control de versiones local
GitHub	Repositorio remoto en línea

Estructura del repositorio

El repositorio se configuró con los siguientes elementos obligatorios:

- Archivo README.md con secciones: Instalación, Uso, Créditos y Licencia.
- Archivo .gitignore que ignora: *.class, *.swp y archivos de proyecto de IntelliJ IDEA.
- Branch principal: master
- Branch de desarrollo: develop.

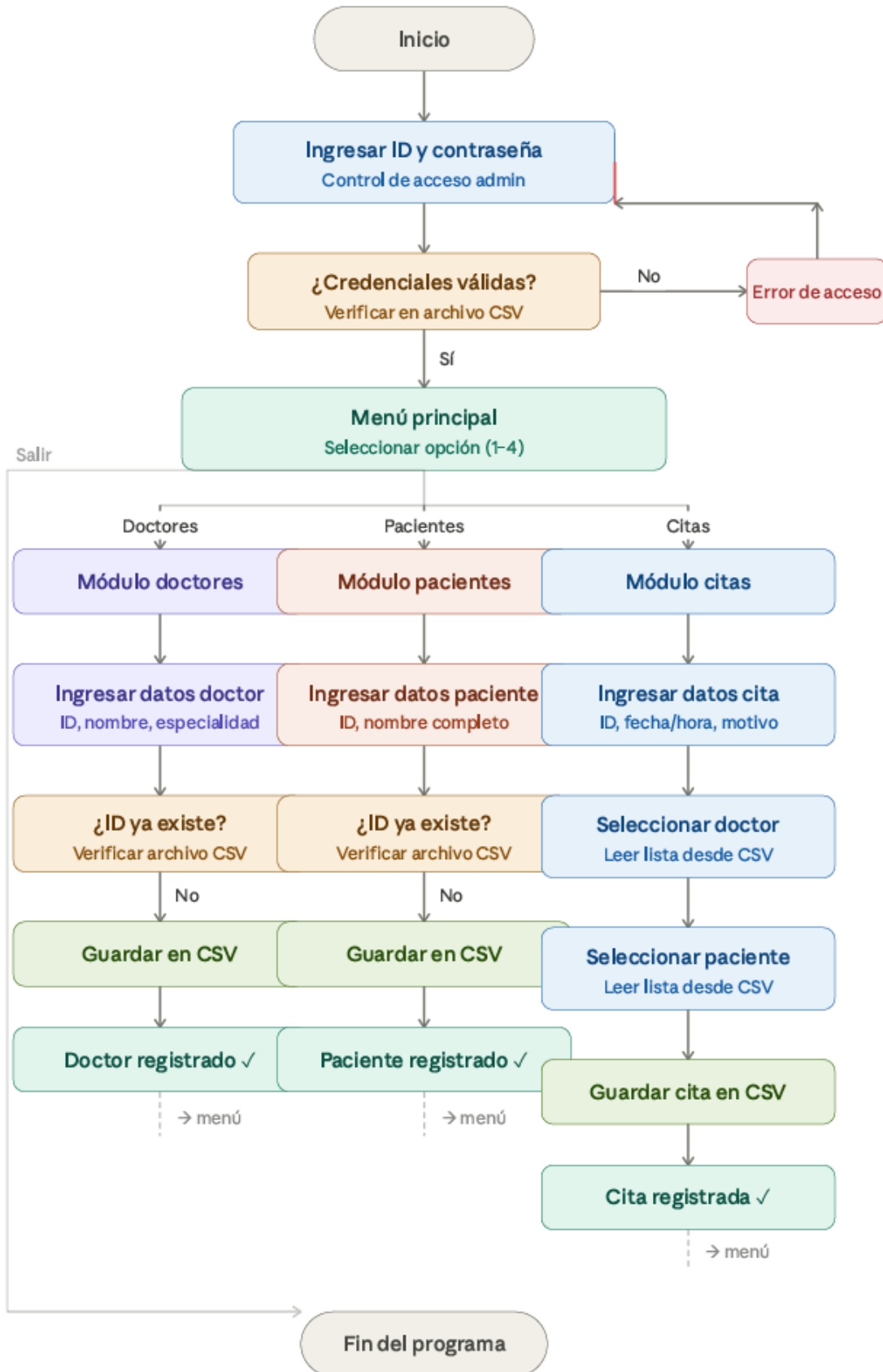
2. Diagrama de Flujo

El diagrama de flujo cubre el ciclo completo del sistema:

- El programa inicia y solicita credenciales al administrador.
- Si las credenciales son incorrectas, muestra error y vuelve a solicitar ingreso.
- Si son correctas, despliega el menú principal con 4 opciones.
- Módulo Doctores: captura ID, nombre y especialidad; verifica duplicado; guarda en CSV.
- Módulo Pacientes: captura ID y nombre completo; verifica duplicado; guarda en CSV.
- Módulo Citas: captura ID, fecha/hora y motivo; selecciona doctor y paciente desde CSV; guarda cita.
- Opción Salir: termina el programa de forma controlada.
- Toda excepción es capturada y mostrada en pantalla sin detener la ejecución.

Leyenda





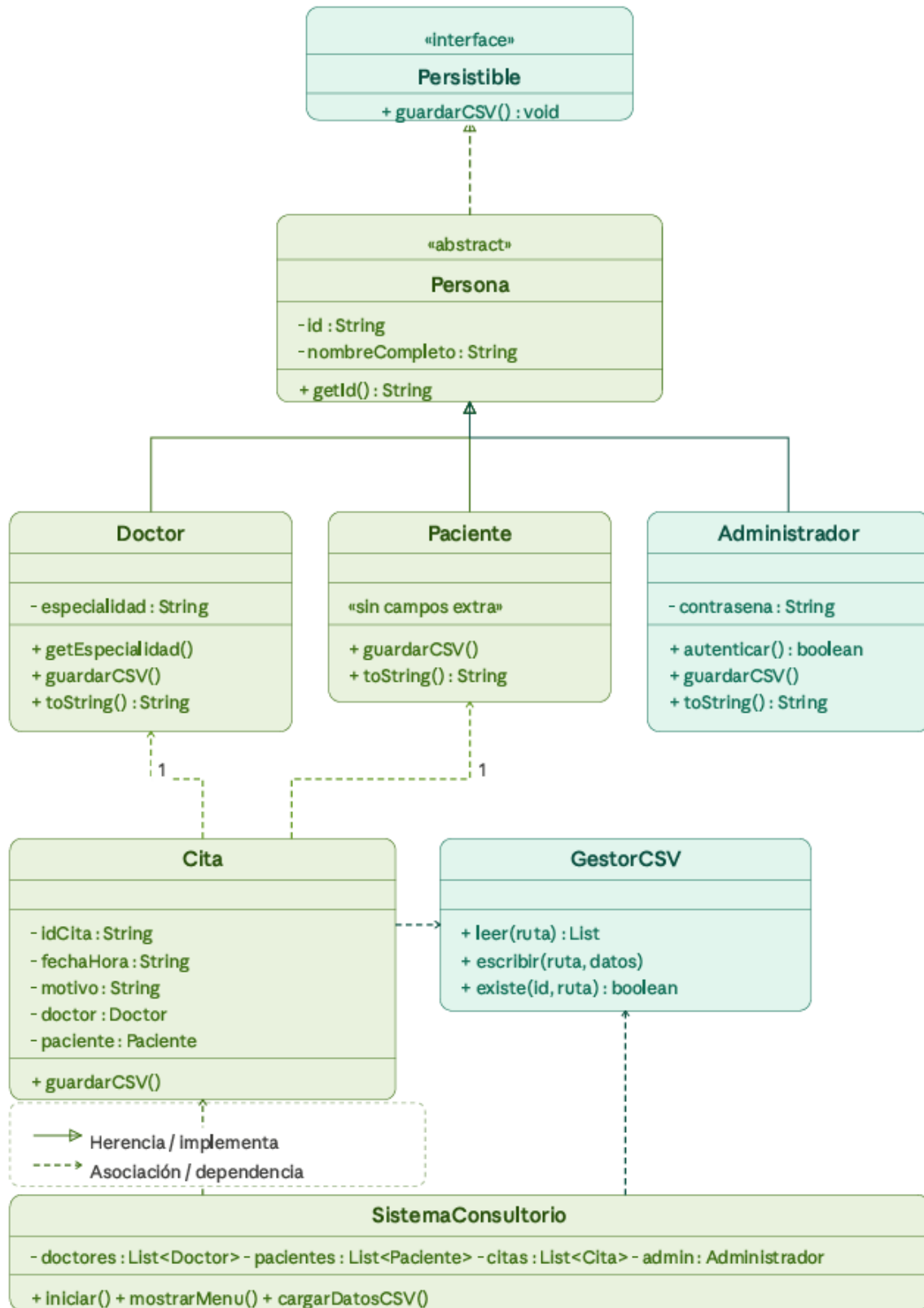
3. Diagrama de Clases

Descripción de clases

Clase	Tipo	Responsabilidad
Persona	Abstracta	Clase base con id y nombreCompleto. Padre de Doctor, Paciente y Administrador.
Doctor	Concreta	Extiende Persona. Agrega especialidad. Implementa Persistible.
Paciente	Concreta	Extiende Persona. Sin campos extra. Implementa Persistible.
Administrador	Concreta	Extiende Persona. Agrega contraseña y método autenticar().
Cita	Concreta	Contiene idCita, fechaHora, motivo, y referencias a Doctor y Paciente.
SistemaConsultorio	Concreta	Clase principal (controlador). Gestiona listas y el flujo del menú.
GestorCSV	Concreta	Utilitaria. Lee y escribe archivos CSV para todas las entidades.
Persistible	Interfaz	Contrato guardarCSV(). Implementada por Doctor, Paciente, Cita y Administrador.

Relaciones entre clases

- Herencia (extends): Doctor → Persona, Paciente → Persona, Administrador → Persona.
- Implementación (implements): Doctor, Paciente, Administrador, Cita → Persistible.
- Asociación: Cita tiene referencia a un Doctor y a un Paciente (1 a 1 cada uno).
- Dependencia: SistemaCitas usa GestorCSV para todas las operaciones de E/S.



4. Pseudocódigo

Programa principal

```

INICIO
    sistema ← nuevo SistemaConsultorio()
    sistema.cargarDatosCSV()
    admin ← sistema.obtenerAdministrador()

    HACER
        MOSTRAR 'Ingresa ID de administrador: '
        idIngresado ← LEER_LINEA()
        MOSTRAR 'Ingresa contraseña: '
        passIngresado ← LEER_LINEA()
        MIENTRAS NO admin.autenticar(idIngresado, passIngresado)
            MOSTRAR 'Credenciales incorrectas. Intenta nuevamente.'

        MOSTRAR '¡Bienvenido, ' + admin.getNombre() + '!'
        sistema.mostrarMenu()
    FIN

```

Menú principal

```

FUNCION mostrarMenu()
    opcion ← 0
    MIENTRAS opcion ≠ 4
        MOSTRAR '1. Gestión de doctores'
        MOSTRAR '2. Gestión de pacientes'
        MOSTRAR '3. Gestión de citas'
        MOSTRAR '4. Salir'
        opcion ← LEER_ENTERO()
        INTENTAR
            SI opcion = 1 → gestionarDoctores()
            SI opcion = 2 → gestionarPacientes()
            SI opcion = 3 → gestionarCitas()
            SI opcion = 4 → MOSTRAR 'Hasta pronto.'
            SINO → MOSTRAR 'Opción inválida.'
        ATRAPAR (excepcion)
            MOSTRAR 'Error: ' + excepcion.mensaje
    FIN MIENTRAS
FIN FUNCION

```

Módulo – Dar de alta doctor

```

FUNCION gestionarDoctores()
    MOSTRAR 'Ingresa ID del doctor: '
    id ← LEER_LINEA()

    SI GestorCSV.existe(id, 'doctores.csv')
        MOSTRAR 'Error: ya existe un doctor con ese ID.'
        RETORNAR
    FIN SI

    MOSTRAR 'Ingresa nombre completo: '

```

```

nombre ← LEER_LINEA()
MOSTRAR 'Ingresa especialidad: '
especialidad ← LEER_LINEA()

doctor ← nuevo Doctor(id, nombre, especialidad)
doctor.guardarCSV()
sistema.agregarDoctor(doctor)
MOSTRAR 'Doctor registrado correctamente.'
FIN FUNCION

```

Módulo – Dar de alta paciente

```

FUNCION gestionarPacientes()
  MOSTRAR 'Ingresa ID del paciente: '
  id ← LEER_LINEA()

  SI GestorCSV.existe(id, 'pacientes.csv')
    MOSTRAR 'Error: ya existe un paciente con ese ID.'
    RETORNAR
  FIN SI

  MOSTRAR 'Ingresa nombre completo: '
  nombre ← LEER_LINEA()

  paciente ← nuevo Paciente(id, nombre)
  paciente.guardarCSV()
  sistema.agregarPaciente(paciente)
  MOSTRAR 'Paciente registrado correctamente.'
FIN FUNCION

```

Módulo – Crear cita

```

FUNCION gestionarCitas()
  MOSTRAR 'Ingresa ID de la cita: '
  id ← LEER_LINEA()

  SI GestorCSV.existe(id, 'citas.csv')
    MOSTRAR 'Error: ya existe una cita con ese ID.'
    RETORNAR
  FIN SI

  MOSTRAR 'Ingresa fecha y hora (yyyy-MM-dd HH:mm): '
  fechaTexto ← LEER_LINEA()
  fechaHora ← LocalDateTime.parse(fechaTexto)

  MOSTRAR 'Ingresa motivo de la cita: '
  motivo ← LEER_LINEA()

  MOSTRAR 'Doctores disponibles:'
  PARA CADA doctor EN sistema.getDoctores()
    MOSTRAR doctor.getId() + ' - ' + doctor.getNombre()
  FIN PARA
  MOSTRAR 'Selecciona ID del doctor: '
  idDoctor ← LEER_LINEA()

```

```

doctor ← sistema.buscarDoctor(idDoctor)

MOSTRAR 'Pacientes disponibles:'
PARA CADA paciente EN sistema.getPacientes()
  MOSTRAR paciente.getId() + ' - ' + paciente.getNombre()
FIN PARA
MOSTRAR 'Selecciona ID del paciente: '
idPaciente ← LEER_LINEA()
paciente ← sistema.buscarPaciente(idPaciente)

cita ← nuevo Cita(id, fechaHora, motivo, doctor, paciente)
cita.guardarCSV()
sistema.agregarCita(cita)
MOSTRAR 'Cita registrada correctamente.'
FIN FUNCION

```

Gestor CSV – Operaciones de archivo

```

FUNCION leer(ruta) → Lista de arreglos de texto
  lineas ← leerArchivo(ruta)
  filas ← lista vacía
  PARA CADA linea EN lineas
    filas.agregar(linea.split(','))
  FIN PARA
  RETORNAR filas
FIN FUNCION

FUNCION escribir(ruta, datos)
  INTENTAR
    abrir archivo(ruta, modo APPEND)
    escribirLinea(datos.unirCon(','))
    cerrar archivo
  ATRAPAR (IOException e)
    MOSTRAR 'Error al escribir: ' + e.mensaje
  FIN INTENTAR
FIN FUNCION

FUNCION existe(id, ruta) → booleano
  filas ← leer(ruta)
  PARA CADA fila EN filas
    SI fila[0] = id → RETORNAR verdadero
  FIN PARA
  RETORNAR falso
FIN FUNCION

```